

Grid Computing for Bag of Tasks Applications

Walfredo Cirne Francisco Brasileiro Jacques Sauvé
Nazareno Andrade Daniel Paranhos Elizeu Santos-Neto Raissa Medeiros
UFCG – Universidade Federal de Campina Grande
{walfredo,danielps,nazareno,elizeu,raissa,fubica,jacques}@dsc.ufcg.edu.br

Bag-of-Tasks applications (those parallel applications whose tasks are independent) are both relevant and amendable for execution on computational grids. In fact, one can argue that Bag-of-Tasks applications are the applications most suited for grids, where communication can easily become a bottleneck for tightly-coupled parallel applications. In spite of such suitability, few users currently execute their Bag-of-Tasks applications on grids. This state of affairs inspires preoccupation. After all, if it is hard to use the grid even with Bag-of-Tasks applications, grids are not going to be of much use. This article investigates this very issue. We identify three key features (automatic access granting, grid working environment, and application scheduling) needed for the execution of Bag-of-Tasks applications on grids, and describe efforts on how to provide such functionality. Unfortunately, however, some practical hurdles make deploying these features in practice much harder than one might think at first. Therefore, we discuss the four major problems (lack of end-to-end connectivity, protocol heterogeneity, security issues, and fault diagnosis) one faces when implementing the proposed functionality in practice.

1 Introduction

Bag-of-Tasks (BoT) applications are those parallel applications whose tasks are independent of each other. Despite their simplicity, BoT applications are used in a variety of scenarios, including data mining, massive searches (such as key breaking), parameter sweeps [1], simulations, fractal calculations, computational biology [32], and computer imaging [27] [28]. Moreover, due to the independence of their tasks, BoT applications can be successfully executed over

widely distributed computational grids, as has been demonstrated by the SETI@home project [3]. In fact, one can argue that BoT applications are the applications most suited for computational grids, where communication can easily become a bottleneck for tightly-coupled parallel applications.

Nevertheless, few users of BoT applications are currently using computational grids, despite the potential dramatic increase in resources grids can bring to bear for problem resolution (since BoT applications are very loosely-coupled, many resources can be useful for a single application). We believe that the lack of effective implementations of important functionality on existing grid middleware plays an important role for this state of affairs.

After three years working on this very problem, trying to provide solutions for real users [4] [13] [15] [25] [26], we claim to have identified the minimal set of functionality required to execute BoT applications on grids, namely: (i) automatic access granting, (ii) suitable grid working environment, and (iii) efficient application scheduling. Unfortunately, no grid infrastructure currently used addresses all three issues in a comprehensive way.

In this paper we analyze current solutions to provide such functionality. We also discuss the difficulties in implementing it. More precisely, we show how the lack of end-to-end connectivity, the difficulty in performing fault diagnosis, protocol heterogeneity, and security concerns create several practical problems for the effective use of grids as execution platforms for BoT applications.

2 Needed Functionality for Running BoT Applications

BoT applications can potentially benefit from a very large number of processors, which would inevitably form a very heterogeneous grid (different computer architectures, different processor speeds, different operating systems, different compilers and libraries, different file system organization, and so on). On the other hand, the user wants to focus on her application, avoiding grid details as much as possible. Therefore, the great challenge in grid computing for BoT applications is to isolate the user from the grid's complexity, while not precluding opportunities for performance enhancement. In order to do so, we claim we need to provide the following features: (i) automatic access granting, (ii) a suitable grid working environment, and (iii) efficient application scheduling.

The first functionality is commonly overseen and normally left out of the scope of current grid middleware, despite the obvious fact that a user can make no use of grid middleware if she has no grid to use. Note that letting the user and/or system administrators "manually" assemble the grid typically produces a grid of somewhat modest size, therefore limiting the gains that can be achieved.

Another important issue, covered by the second functionality, is that of providing the user with a set of abstractions that enable her to conveniently use the grid, in the same way that files and processes make it convenient to use a single computer. This *grid working environment* should provide a common denominator that users can rely upon when using and programming for grids, despite differences in the configuration of the multiple resources that comprise the grid.

Finally, the user wants her application to execute without long delays. Unfortunately, efficiently scheduling BoT applications on grids is not as easy as it might look at first. Good scheduling requires accurate information about the tasks that compose the application and the capabilities of grid resources. Requiring information about tasks (such as expected execution time) is a distraction for the user and typically results in poor quality information [14]. On the other hand, automatically gathering information about grid resources (such as speed and load) is often difficult due to the grid's distributed and multi-institutional nature. Moreover, many interesting BoT applications are also data intensive. Therefore, good scheduling for BoT applications on grids must consider data transfers besides the traditional processing time.

2.1 Automatic access granting

An application needs resources to run. As obvious as this sounds, current grid computing solutions do not provide a mechanism for the user to obtain access to new resources. This problem is typically removed from the scope of the middleware. In Globus, for example, a resource administrator must manually map the grid-wide user-id of each user that can access the resource to a local user-id [16]. In MyGrid, as another example, the user herself assembles her grid. That is, one's MyGrid is composed of "all resources one has access to" [15].

It is easy to understand why current grid solutions avoid the issue of obtaining automatic access to resources. Granting access is a policy decision. Moreover, automatically (i.e. with no human intervention) granting access is a potential security threat. However, automatic access granting is very important for BoT applications because their very loosely coupled nature enable them to benefit from large amounts of resources. If the access to each resource in the grid has to be negotiated and configured by humans, there is a great limitation on the benefits a BoT application can attain from the grid.

Gaining automatic access is still an open question. There have been efforts in grid economy [2] [9] that aim to solve this very problem. However, grid economy solutions demand non-trivial e-cash and e-banking infrastructure to be deployed. Moreover, they require QoS guarantees (clients expect to receive what

they “pay” for). As a result, it is currently very hard to implement grid economy proposals in practice.

Note, however, that grid economy schemes aim to solve the general problem of gaining automatic access. Since BoT applications can run well with no QoS guarantees (for example, when using idle cycles [23]), we believe we can solve a simpler instance of gaining automatic access that can be implemented today and still useful for BoT applications.

In fact, we are experimenting with these ideas in OurGrid [4]. OurGrid is a peer-to-peer resource sharing system targeted to BoT applications, where each peer is a site (i.e. a set of machines in a single administrative domain). OurGrid works as a *network of favors*. That is, each site offers access to its idle resources to the community. In return, when there is work that exceeds local capacity, a site expects to gain access to the idle resources of other participants. OurGrid aims to allow users of BoT applications to easily obtain access and use computational resources, dynamically forming an on-demand, large-scale grid.

Since participation in an OurGrid community is voluntary, in order to motivate resource sharing, the network of favors is designed (i) to promote equity (i.e., the resources a site obtained from the grid should be proportional to the resources donated to the community), and (ii) to prioritize the sites that helped the community the most (in the sense that they have donated more than they have consumed).

In the network of favors, giving access to a resource is a favor. As such, it is expected that the consumer incurs in “debt” with the owner of the consumed resources. Every site in the system keeps track of a local balance of the past interactions with each other known site. This balance is used to prioritize sites with more credit when arbitrating conflicting requests. More precisely, using all known sites’ balances, each participant maintains a ranking of all known sites. This ranking is updated on providing or consuming favors. The quantification of each favor’s value is done locally and independently – negotiations and agreements aren’t used – thus affecting only decisions of future resource allocations made by the local site. As the sites in the system ask each other favors, they gradually discover which participants are able to reciprocate their favors, and prioritize them, based on their debt or credit.

As a consequence, while a participant prioritizes those who cooperate with him in satisfactory ways, it marginalizes the sites that, for any reason, do not reciprocate the favors satisfactorily. The non-retribution can happen for many reasons, such as service or communication network failures, the absence of the desired service in the site, or the utilization of the desired service locally or by other

users at the moment of the request. Free-riding sites may even choose not to reciprocate favors. In any case, the non-retribution of the favors gradually diminishes the probability of the site to access the grid's resources.

The rationale behind OurGrid design is the reciprocation of favors to the consumers that a site is in debt with. If a participant does not act in this way, it is given gradually less priority by the community, as its debt grows. Some preliminary results suggest that our design meets its goals [4]. Due to its decentralized and autonomous nature, characterizing the behavior of an OurGrid community is both very important and quite challenging. Therefore, we initially based our analysis on a simplified scenario meant to capture the key features of OurGrid, namely the system-wide behavior of the network of favors and the contention for finite resources. The simplification consists of grouping resource consumption into *turns*. In a turn, each site is either a provider or a consumer. If a site is a consumer, it tries to consume all available resources. If a site is a provider, it tries to donate all resources it owns to the current turn consumers. Under this model, every site owns a variable amount r of identical resources and has a fixed probability p of being a provider in a given turn. Although simplified, this model has already allowed us to draw some interesting conclusions about the network of favors. Here we show some of these results. A more detailed discussion is presented in [4].

Two properties were observed through this model: the equity and prioritization of resource access. These properties are gauged by two metrics: the Favor Ratio (FR) and the Resource Gain (RG), respectively. Favor Ratio FR is defined by the ratio of the accumulated amount of resources gained from the grid (note that this excludes the local resources consumed) to the accumulated amount of resources donated to the grid. Attaining equity thus mean $FR = 1$ (i.e. the amount of donated and received resources is the same). The Resource Gain RG of a site is the ratio of the accumulated amount of resources used by it (both local and from the grid) to the accumulated amount of local resources it has used. RG represents the priority obtained by a site in the sense that the greater the RG, the greater the contribution of the grid to the site. That is, the greater the RG, the higher the priority is.

The histogram in Figure 1 shows the distribution of Favor Ratio FR for a 100-site community after 3000 turns. As can be seen, FR converges to 1. This denotes equity as the sites where able to get back from the community as many resources as they donated. This convergence happened in all the simulated scenarios where there was enough competition to observe the contention for resources.

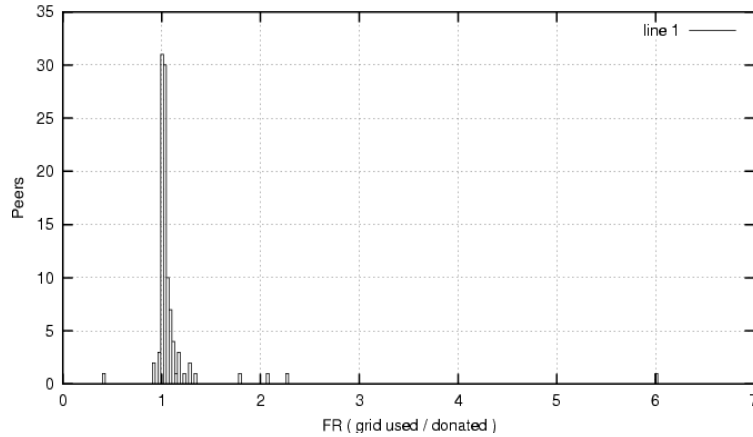


Figure 1 – Histogram of Favor Ratio FR values for a 100-site community on round 3000

Figure 2 shows Resource Gain RG for three sites with different ρ values in a 10-site community. As such, it shows how a site is prioritized as it donates more resources to the community. Consequently, the site that provided more resources is the site with the greatest RG. Note that, for the site with the greatest ρ , RG goes out of scale in its first request, after it spent some turns providing. This gives us the almost vertical solid line in the graph.

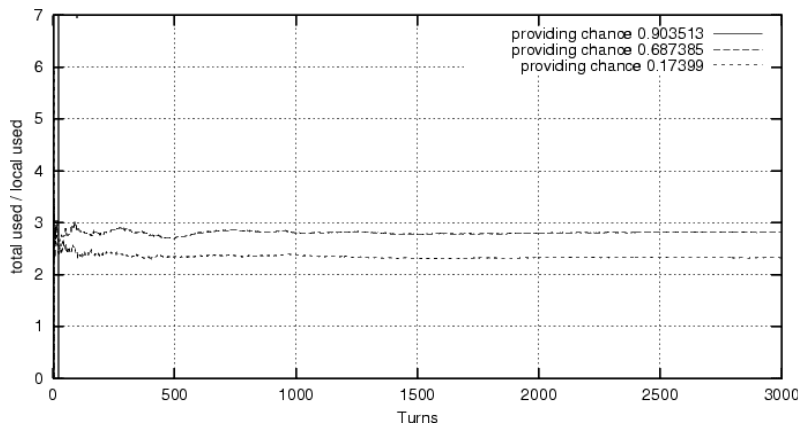


Figure 2 – Resource Gain RG values for 3 sites in a 10-site community

These results suggest that our approach is promising. We are currently addressing some other important aspects of OurGrid communities (security, accounting, more realistic scenarios) and initiating development of a prototype of the system.

2.2 Grid Working Environment

There are two alternatives for providing a grid working environment. As in Condor [23], one can intercept system calls performed on grid machines and re-

direct them to the user's machine, creating the illusion that all tasks run on the user's machine. Or, as in MyGrid, one can devise abstractions that enable the user to express the computation to be done with no knowledge about the grid machines.

Condor Working Environment

Condor is a system that aims to offer large amounts of computational power over medium and long time ranges (days to weeks) using idle processors on a network [23]. In order to provide the greatest possible quantity of resources, no assumption is made that the idle machine used to execute a task mounts the same file systems mounted by the home machine. The Condor user has the illusion that submitted tasks run locally on the user's home machine. This is achieved by having Condor itself redirecting system calls performed on the remote machine to the user's home machine, as shown in Figure 3. This redirection is implemented by Condor's redirecting library. This library offers standard file manipulation routines (i.e., `open()`, `read()`, `write()`, etc.). The library decides whether the call should be performed locally (while accessing a temporary file, for example) or be redirected to the home machine (while accessing a file under `/home`, for example). For local access, a standard system call is used. For remote access, a remote procedure call (RPC) is performed to the Condor server running on the home machine.

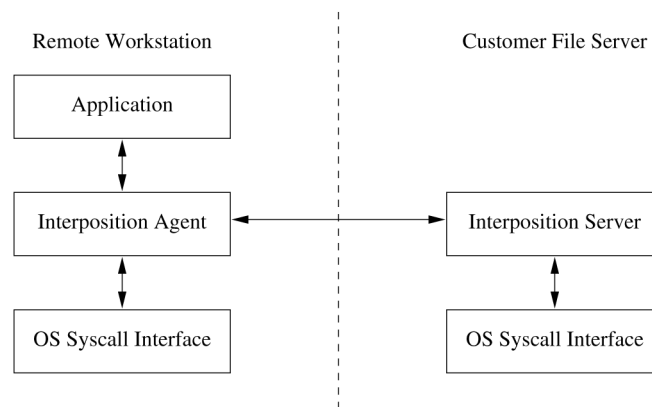


Figure 3 – Condor's system call redirection [8]

MyGrid Working Environment

A MyGrid application is comprised of a number of independent tasks. Each MyGrid task is formed by *initial*, *grid*, and *final* subtasks, which are executed sequentially in this order. The initial subtask runs on the user's *home machine* and is meant to set up the task's environment by, for instance, transferring the input data to the grid machine. The grid subtask runs on a (remote) grid ma-

chine and performs the computation per se. The final subtask is typically used to collect the task's results back to the home machine.

MyGrid abstractions allow for writing the subtasks without knowing details about the grid machines used (such as file systems organization). The abstractions provided to that effect are *playpen*, *file transfer*, and *mirror*. *Playpens* provide temporary disk space independently of the local file system arrangements of a given grid machine. Playpens are directories created automatically to serve as the working directory of the grid subtask. A grid subtask can also refer to its playpen via the `$PLAYPEN` environment variable. MyGrid creates the playpen in a file system that can hold the amount of data specified by the user as the task's *playpen size*. (If there is no such file system in a given grid machine, the task cannot be scheduled to this machine.)

File transfer allows for sending files between grid machines and the home machine. They are typically used for the initial subtask to send input data to the playpen, and for the final subtask to collect output data from the playpen. In order to ease writing the initial and final subtasks, MyGrid automatically defines the environment variables `$PROC`, `$PLAYPEN` and `$TASK`. They respectively contain the grid machine chosen to run the task, the directory created as the playpen, and the unique task number.

Mirroring enables replication of home machine files on grid machines. Mirrored files are put in the directory indicated by the `$MIRROR` environment variable, which is defined by MyGrid taking into account the grid machine's file system organization. Therefore, a grid subtask refers to mirrored file *F* through `$MIRROR/F`, without having to know details about the grid machine's file system. Mirroring is useful for distributing files that are going to be used more than once, such as program binaries. Mirroring is implemented efficiently by using the modification date and a hash of mirrored files, avoiding unnecessary file transfers.

For example, suppose we want to run the binary task, which has the file `INPUT` as input and the file `OUTPUT` as output. The initial subtask would then be:

```
mg-services mirror $PROC task
mg-services put $PROC INPUT $PLAYPEN
```

The grid subtask would be simply:

```
task < INPUT > OUTPUT
```

And the final subtask would collect `OUTPUT` to the `results` directory, renaming the file by appending the unique task number to its name.

```
mg-services get $PROC $PLAYPEN/OUTPUT results/OUTPUT-$TASK
```

Appending the task number to a file is useful for the quite common case where the tasks that compose the application produce output with the same name. Appending the task number ensures the uniqueness of each output file in the home machine.

Comparing the alternatives

The alternatives presented above represent different trade-offs in the design space of grid working environments. Condor's working environment approach is simpler for the user to understand, but requires relinking the application with Condor's redirection library. MyGrid's working environment approach is not complicated, but it does require the user to learn a few new concepts. On the other hand, MyGrid's approach does not require end-to-end communication between the user machine and the grid machine (see Section 3.2), leading to simpler deployment for MyGrid. Which approach has the best performance is heavily dependent on the application. If a MyGrid application transfers a large file to use only part of it, performance would be better with Condor. If a Condor application accesses the same file many times, MyGrid would likely do better. Finally, notice that Condor's approach implies that a write is immediately propagated to the user's machine, whereas in MyGrid results go to the user's machine only when tasks finish. This eases fault recovery in MyGrid (simply resubmit the task, without worrying about side effects) and enables task replication, an important issue for efficient scheduling as discussed next.

2.3 Application Scheduling

Scheduling BoT applications may appear as a very simple task. However, grids introduce two issues that complicate matters. First, it is typically difficult to obtain information about the application (such as estimated execution time) and the grid (processor speed, network topology, and so on). Second, since many important BoT applications are also data-intensive applications, considering data transfers is paramount to achieve good performance. But, of course, processing time still has to be considered. Such as double-goal scheduling (data and computation) is non trivial. Here we present solutions that tackle both problems individually. However, a solution that addresses both problems simultaneously is still missing.

Scheduling with no information

As mentioned before, in order to perform well, scheduling algorithms need to be fed with comprehensive and accurate information about the tasks that compose the application and the capabilities of grid resources. Normally, users have neither the will nor the means to provide such information with the quality required [14]. Further, mechanisms to automatically acquire this information do

not perform well (both in scope and accuracy) due to the grid's distributed and multi-institutional nature.

An alternative is to use a scheduler that does not rely on information *about* tasks or resources, such as *Workqueue*. *Workqueue* only uses information absolutely necessary to schedule the application, namely the list of tasks to run, and the list of processors available to execute the tasks. In *Workqueue*, yet-to-execute tasks are chosen in an arbitrary order and sent to the processors, as soon as they become available. After the completion of a task, the processor sends back the results and the scheduler assigns a new task to the processor. Unfortunately, knowledge-free schedulers (such as *Workqueue*) do not attain performance comparable to schedulers based on full knowledge about the environment (provided that these schedulers are fed with good information) [26].

We developed the *Workqueue with Replication* (WQR) algorithm to deal with this problem. WQR delivers good performance without using any kind of information about the resources or tasks. WQR is an eager scheduler [7]. Initially, WQR behaves as the conventional *Workqueue*. The difference appears when there are no more tasks to execute. At this time, a machine that finishes its tasks would become idle during the rest of the application execution in *Workqueue*. Using replication, such a machine is assigned to execute a replica of an unfinished task. Tasks are replicated until a predefined maximum number of replicas is achieved. When a task is replicated, the first replica that finishes is considered as the valid execution of the task and the other replicas are cancelled. Of course, WQR assumes that tasks are idempotent, i.e. can be re-executed with no side effects. Since MyGrid's abstractions (see Section 2.2) assure no side-effects until the end of the task execution, WQR was easily incorporated to MyGrid. A way to think about WQR is that it trades additional CPU cycles for the need of information about the grid and the application.

We investigated the performance of WQR under a variety of scenarios, in which we varied the granularity of the application (how many tasks compose the application), the heterogeneity of the application, and the heterogeneity of the grid [26].

Table 1 summarizes the results of 7,885 simulations. Suffrage and Dynamic FPLTF (Fastest Processor to Largest Task First) are known scheduling algorithms that were fed with perfect knowledge in the simulations. The qualifier to WQR (i.e. 2x, 3x, 4x) denotes the maximum replication allowed. For example, WQR 2x allows only 2 copies of each task.

Table 1 – WQR simulation results

		Sufferage	Dynamic FPLTF	Workqueue	WQR 2x	WQR 3x	WQR 4x
Execution time (sec)	Mean	13530.26	12901.78	23066.99	12835.70	12123.66	11652.80
	Std Dev	9556.55	9714.08	32655.85	10739.50	9434.70	8603.06
Wasted CPU (%)	Mean	N/A	N/A	N/A	23.55	36.32	48.87
	Std. Dev	N/A	N/A	N/A	22.29	34.79	48.94

Overall, the performance of WQR appeared to be equivalent to solutions that have perfect knowledge about the environment (which it is not feasible to obtain in practice), even when we were limited to two replicas of each task. The wasted CPU varied in average from 23.5% (when using only 2 replicas) to 48.9% (when using 4 replicas, the maximum tried).

Note also that the high values of the standard deviation suggest a great deal of variability in the results. In fact, we found that application granularity has a strong impact on the results. In our experiments, both the application size and the grid power were constant, thus application granularity is given by the relation of machines to tasks. As can be seen in Figure 4, WQR attains good performance, except for applications with very large granularity (where there are more machines than tasks). The difficulty faced by WQR when there are more machines than tasks is that the application execution takes only one “round” of tasks to processors assignments. Therefore, assigning a large task to a slow machine has great impact on the application execution time. Algorithms based on knowledge about the application and the grid (Sufferage and Dynamic FPLTF) can avoid such bad assignments. Replication helps (as can be seen comparing WQR to Workqueue) but not enough to overcome the problem totally. Not surprisingly, much more cycles are wasted when only a single round to tasks to processors assignments, as shown by Figure 5. Consequently, when there are more machines than tasks, one might want to limit cycle waste by limiting replication to 2. Performance is still reasonable (although not as good as that achieved by scheduling algorithms with perfect knowledge of the application and the grid). Application and grid heterogeneity also influence WQR performance, but to a smaller degree. We refer the reader to [26] for a complete performance analysis of WQR.

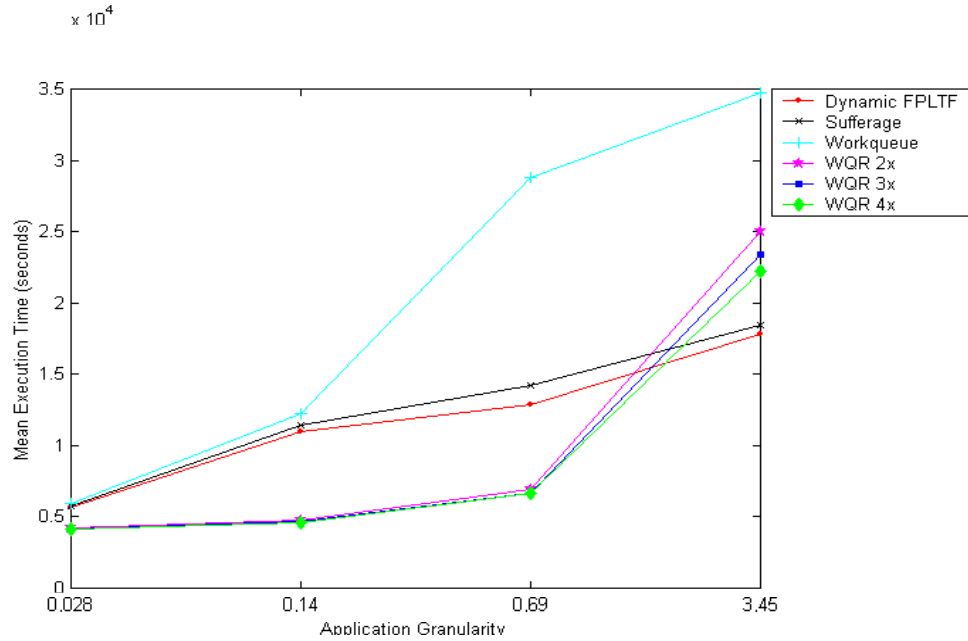


Figure 4 – Execution time by application granularity

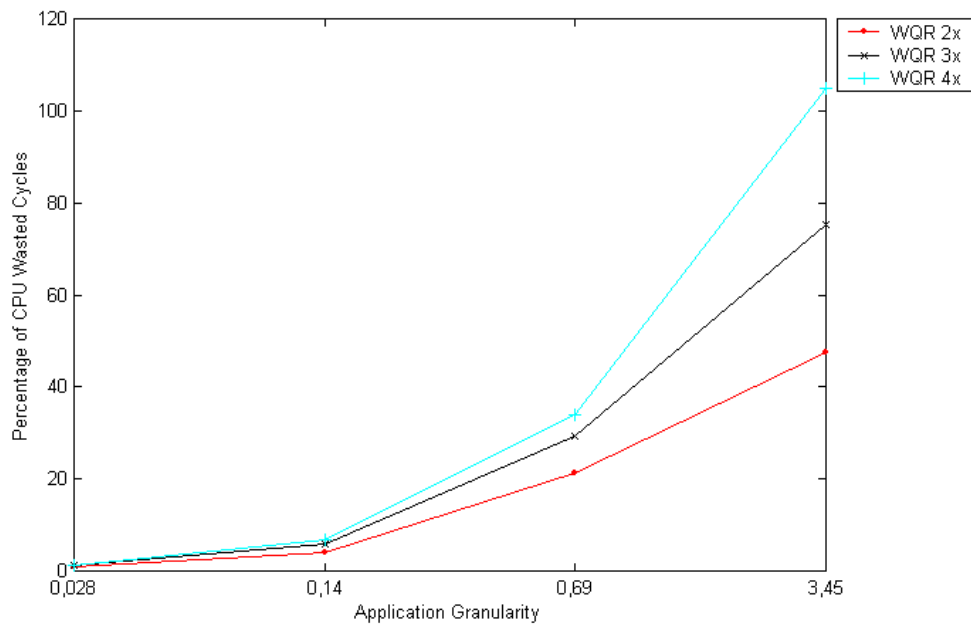


Figure 5 – Wasted cycles by application granularity

There are other schedulers for BoT applications on grids, such as those found in APST [11] [12] and Nimrod/G [1]. However, they are more specialized than WQR in the sense that they target parameter-sweep applications, a subclass of BoT applications. Also, both APST and Nimrod/G require much more information than WQR for scheduling. On the other hand, APST takes data trans-

fer into account when scheduling, whereas WQR assumes tasks to be CPU-bound. We are currently working on WQR to overcome this limitation.

Scheduling taking data placement into account

The APST scheduling solution is based on the XSufferage heuristic described in [10]. XSufferage is an extension of the Sufferage scheduling heuristic. Sufferage schedules a task that would “suffer” more hadn’t it being scheduled in that processor. In order to compute the sufferage value of a task, one must compute the expected *completion time* of the task in all processors. The completion time is the time that an individual task takes to execute on a given processor and is estimated using dynamic environment and application information. The sufferage value is then the completion time on the second fastest processor minus the completion time on the fastest processor. It captures how much a task would suffer if it doesn’t get assigned to the processor that can run it faster.

The purpose of XSufferage is scheduling applications taking into account the data transfers. XSufferage assumes that the processors in the grid are grouped in sites, which is very reasonable. XSufferage then replaces completion time by *site-level completion time*. The site-level completion time is the minimum completion time the task can achieve using one of the site’s processors. Note that the value of site-level completion time considers the costs of transferring data to/from the site. After the calculations, the task with the greatest site-level sufferage value is allocated. The process continues until all tasks are allocated. Figure 6 shows XSufferage pseudo-code.

```

while there are TASKS to schedule do
  for each TASK that comprise application and not scheduled do
    for each cluster that comprise the grid do
      determine the cluster-level CT value
      determine the processor that present the minimum CT in the cluster
    end
    determine the best and second best cluster-level CT
    calculate the cluster-level Sufferage
  end
  schedule the task with greater cluster-level Sufferage
end

```

Figure 6 – XSufferage Algorithm

The idea behind that heuristic is to take advantage of previously executed data transfers to reduce the execution time of parameter sweep applications. In other words, the idea is that a machine is assigned to a task that would “suffer” the most if that machine were not assigned to it. For data intensive applications, the proximity of input data to the task is a decisive factor for application perform-

ance. If the input data of a task is not stored at the site where the task is assigned, performance is degraded. And, indeed, the performance evaluation presented in [10] show that XSufferage performs much better than scheduler that are not aware of data placement.

However, it is important to note that XSufferage uses lots of dynamic information about the grid environment and the application. As discussed above, this kind of information is sometimes difficult to obtain and often unavailable due to administrative restrictions, hindering the application of this solution.

3 Practical Problems

The functionalities described in the previous section are enough to enable convenient grid computing for BoT applications. However, implementing and using such functionality leads to four practical problems that hinder or limit the scope of grid computing for BoT applications. These problems are (i) protocol heterogeneity, (ii) lack of end-to-end connectivity, (iii) establishing trust and dealing with security, and (iv) fault diagnosis. As we shall see, these problems are not conceptual. They are policy-driven and/or arise due the intrinsic wide distribution and lack of central administration of the grid. They may even disappear with the evolution of the technology. Nevertheless, they do hinder using computational grids today.

3.1 Protocol Heterogeneity

We have known how to build interoperable computer systems for at least two decades. The solution is simple and elegant: standardize the protocols. It is no surprise that most grid solutions take the same approach. Unfortunately, grid protocols have yet to be widely accepted. That is, an implementation of the standard protocol is frequently not available (e.g. Globus GRAM is not installed) although other ways of obtaining the same service exist (e.g. `ssh` is installed).

This issue is a serious one for BoT applications because the needed services are often available in a way or another, but the “standard” services (e.g. Globus) are not. Again, since BoT applications can potentially benefit from as many resources as there are available, this limitation of scope is highly undesirable. As an aside, some claim that this will change and that we will eventually have a standard grid implementation on every machine. This may very well be; but, until this future materializes, the multitude of protocols available for equivalent services is a serious limitation for grid computing of BoT applications.

Equivalent solutions to this problem have independently appeared in APST [11] [12] and MyGrid [15] [25] (and probably other projects). It consists of the *local virtualization* of the services needed. That is, there is a *local* standard

interface for the services needed, with multiple implementations for the different protocols. Since the solutions are similar in nature, we will describe only MyGrid's solution. MyGrid defines the *Processor Interface* as the minimal set of services that must be available on a machine for it to be used as a grid machine. These services are:

Table 2 – Processor Interface

Service
Task start-up on a grid machine (remote execution)
Cancellation of a running task
File transfer from the grid machine to the home machine
File transfer from the home machine to the grid machine

As illustrated by Figure 7, there can be many ways to implement the Processor Interface. One way to implement the Processor Interface lets the user furnish MyGrid with scripts that implement the four services listed in Table 2. In this case, MyGrid uses the *Grid Script* module to access the machine. Note that Grid Scripts enable the user to inform MyGrid on how to access a given machine in a very flexible way. This is the technical basis for MyGrid's bold claim of "running the user's application on whatever resources she has access to" [15]. As long as the user is able to translate "having access to a given machine" into "providing scripts that encapsulate such access", MyGrid will be able to use the machine.

Other ways to implement the Processor Interface rely on access methods that are known to MyGrid. For example, we are currently implementing *MyGrid's Globus Proxy*. The idea is that if a grid machine can be accessed through Globus' GSI, GRAM, and GridFTP [20], then the user does not need to supply scripts, but simply indicates that the access is to be done via MyGrid's Globus Proxy. MyGrid also provides its own access mechanism, called *User Agent*. The User Agent is useful when no service that implements the operations described by the Processor Interface (see Table 2) is available.

Of course, the idea of achieving interoperability via local virtualization can bring much greater benefits if the local interface is made standard. For example, say that some version of the Processor Interface is made standard. This would avoid the duplication of efforts in MyGrid and APST (and probably other efforts). One can even think of common development of an implementation of a standard local virtualization (a la open source efforts), mutually benefiting many related projects.

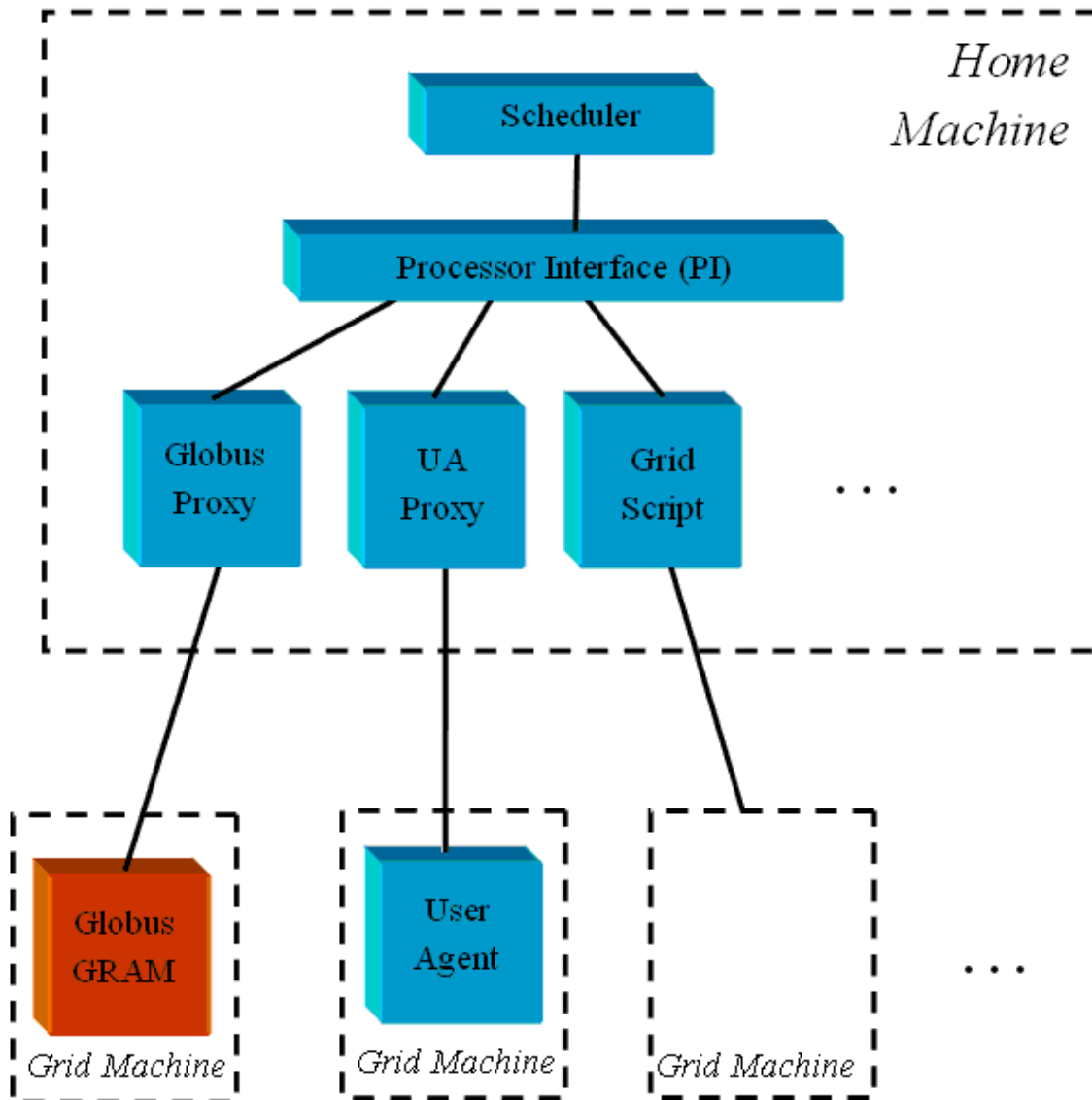


Figure 7 – Implementations of the Processor Interface

3.2 Lack of End-to-End Connectivity

Communication restrictions are a fact in today's Internet [29], where firewalls and private IP addresses are commonplace. Historically, such restrictions have come about due to fear of attack and the lack of IPv4 addresses. Although with IPv6 addresses are no longer scarce, many organizations feel more secure by cutting itself off from direct communication to the Internet, while providing policed ways (firewalls, proxies) of establishing communication, albeit indirectly. As a result, direct IP communication – and hence, direct TCP connections – between two machines meant to be part of the same grid is frequently not possible, with an obvious negative impact on grid computing.

The standard way to deal with this problem is via relaying. Relaying can be application independent [29] or may be specific to an application. For example, MyGrid uses an application-specific relay called *Processor Gateway*. It runs on a machine accessible to both home and grid machines (e.g. in a NAT converter) and forwards Processor Interface services requests (see Table 2).

The problem with relaying (both application dependent and independent) is that it typically requires manual configuration, involving system administrators. Therefore, it poses a problem similar to automatic access granting (see Section 2.1) in the sense that it can limit what resources can be used, seriously restricting the performance improvement that could be attained by a BoT application on the grid.

3.3 Security Issues

BoT applications can take advantage of huge grids. These are normally formed by resources distributed over many administrative domains. Crossing administrative domains immediately gives rise to security issues which, considered in their entirety, can be quite complex. We will briefly consider four security issues that should be considered in any grid, namely establishing trust, maintaining message integrity and confidentiality, establishing and ensuring policy and privacy issues. We then explore issues that are particularly relevant for BoT applications on grids. Finally, some solutions are presented.

Security at large

When two machines exchange messages, one to receive a service, the other to provide it, each has a trust issue to deal with. Should the provider perform the action being asked by the requester? In its turn, should the requester believe that the correct provider is actually performing the work? In short, the communicating parties must decide how much they *trust* each other. Three aspects of trust can be identified. The first is a *social aspect* by which human parties (maybe on behalf of organizations) decide that they will trust each other. When you open an account for Ann on a company's computer, you and Ann are making such a decision and it is inherently non-technical in nature. Secondly, one must establish trust relationships in a technical way. This occurs, for example, when you configure your grid middleware to trust certificates signed by a particular Certificate Authority. Finally, identification and authentication must be performed prior to secure message exchange. Each side can make claims (having a particular name, for example), which should be checked by the other side. In general, this aspect of trust is established by having the two parties exchange security credentials (either directly or indirectly).

Although the technical aspects of trust (establishing trust relationships and performing authentication) seem to be simple requirements to fulfill, several issues may make them rather more complex. Trust between two parties can be established directly (say, by supplying a name/password pair known by each side), or may be brokered (when a party trusts another by trusting a third which vouches for the second). For large-scale systems, a proper solution would have to allow for delegation of credentials, establishment of trust relationships in a federated environment, etc. These additional issues are much less easy to implement.

Having established trust, the attachment of proper security tokens to a message allows message confidentiality (cryptography) and integrity (through message digests and digital signatures, say).

Policy issues are wide ranging and determine what claims a provider will require from the requester to establish trust (see above) and are also used to determine what each party can do (authorization).

A privacy solution allows requesters to state their privacy preferences and also allows organizations to publish privacy practice statements. It should also be possible to check that privacy requirements are matched by published policies enabling message exchange to occur. Additionally, a party may require that incoming requests make claims about the requester's adherence to these policies.

Security for BoT applications

Having briefly outlined the range of issues spanned by security, we may now consider the special case of BoT applications executed on a grid. Although security issues exist in any grid, two competing forces act in contrary directions with particular sharpness in the case of BoT applications. On the one hand, the decoupled nature of BoT applications results in greater incentive to gather up and use all available resources. However, using all possible resources may result in using under doubtful administrative control. Naturally, security becomes more of a problem when one uses resources available under doubtful administrative control. Take the case of SETI@home [3], where millions of computers volunteer into a grid, as an extreme case of "doubtful administrative control". How can SETI@home trust that the provider (any host on the net) will actually perform the required computation rather than simply returning random answers? This is the social aspect of trust in its extreme form.

Assuring the user that resources indeed perform the computation assigned to them is only half of the story. The viewpoint of the resource provider must also be taken into account. In many cases resource providers are organizations such as universities and research centers that have a large amount of com-

putational resources but intend to grant access to only a subset of its resources to grid users. It is important to emphasize that, in order to provide resources for the grid, an organization must have a reasonable assurance that its policies will be correctly enforced. When considering the enforcement of internal policies, the use of authentication and sandboxing are mandatory. The former is important to ensure that only authorized users can access the resource, and latter is important to guarantee that only those resources reserved for the grid will be used. We thus believe that the grid middleware should provide the adequate security support to the enforcement of internal policies, otherwise an organization may decide not to grant access to its resources.

Note that using grid middleware without good sandboxing impacts not only on the security of the grid infrastructure, but also on security of the organization as a whole. This is because nowadays much of the security effort takes place at firewalls. But, when an organization allows for a guest BoT application to run without sandboxing, an attack may be launched from inside the network, completely bypassing the firewalls. In order to remain safe when the grid middleware offers no guarantees on what an application can do, an organization will have to make use of other mechanisms, such as the setup of VLANs and the reconfiguration of firewalls, in order to isolate the subset of the organization's resources that can be used by grid applications. In short, the subset of resources to be used as part of the grid must be segregated from the organization's internal network. The additional administrative burden imposed by such modifications may well be unacceptable by many organizations. As a consequence, these organizations may decide not to join the grid.

The state of the art

As can readily be seen by the brief discussion above, fully dealing with security issues is a huge task. Solutions are difficult to design, since they must be general enough to solve the problem, must age well over time as technologies change, and must allow existing partial security solutions (Kerberos, for example) to be used in a more general framework. As a result, most grid computing middleware ignore the issues, preferring to depend on external mechanisms to provide whatever security is desired by the user.

Nevertheless, some efforts do deal with some of the issue outlined above. Globus Security Infrastructure (GSI) aims to provide secure authentication and delegation services [19] [20], on the top of which one would build higher-level security solutions, encompassing authorization and security policies. SETI@home uses task replication as a simple means of dodging some trust issues (trusting that the service provider actually performs the work rather than

providing bogus answers) [3]. The obvious drawback in task replication is the waste of resources, since each task must be computed at least twice. Sarmiento et al proposed to reduce the replication cost by replicating only a subset of the tasks (as in random auditing), which combined with the provider history may suffice to trust on the results [30]. From the resource provider viewpoint, sandboxing has been used to protect the provider from delinquent application behavior [17] [22].

3.4 Fault Diagnosis

In general, distributed systems are more prone to failures than centralized ones. Grids are subject to them even more, since in such environments there are potentially thousands of resources, services and applications that need to interact. Since these elements are extremely heterogeneous, there are many failure possibilities, including not only independent failures of each element, but also those resulting from interactions between them (for example, a task fails because the browser version in a specific grid resource does not support the desired the Java version). Moreover, machines may be disconnected from the grid due to network partitions or process suspension/abortion in remote machines to prioritize local computation. Such situations cause non-availability of the processing service, characterizing failure situations.

Some solutions for grid fault handling have been proposed (e.g. [5] [6] [26] [31] [33] [34]). They have to deal with complex failure scenarios and need to take into account a number of issues associated with inherent grid characteristics. Strong scalability requirements, the ability to deal with highly heterogeneous resources and the capacity of dynamic adaptation must be considered. These solutions notwithstanding, a survey with grid users shows that they are unhappy with the current state of affairs [24]. The main complaint is that users have to be highly involved in diagnosing failures. In fact, detecting that something is wrong is a relatively simple task (in general, symptoms are quickly identified). Difficulties arise the root cause of the problem needs to be identified, i.e., to *diagnose a fault* in a very complex and heterogeneous environment such as a computational grid.

The first difficulty is to understand what is really happening. Most solutions available (e.g. [5] [6] [31] [33]) are more concerned with grid monitoring and information collecting – functions used not only for fault detection but also for performance analysis, capacity planning and resource allocation – than with grid fault treatment itself. Frequently, lots of information (i.e. logs) about the grid is available, but scanning it to discover what is really happening is next to impossible, especially if the problem is related to middleware and configuration failures.

(Components that provide a nice abstraction when working expose all the gory details when broken.)

This happens because recognizing deviations in any system behavior requires an understanding of its correct functioning. In a grid context, this translates into understanding the detailed functioning of the different technologies that compose it. When failures occur and the transparency provided by the middleware is compromised, the user needs to drill down to lower level of abstractions in order to locate and diagnose faults. This implies dealing with extremely heterogeneous technologies in terms of middleware, operating systems and hardware. There is thus a huge cognitive barrier between the gathering and the interpretation of collected data, i.e., between fault detection and the fault diagnosis.

In order to solve the cognitive problem that no one is going to know all details of a grid to individually diagnosis and fix failures, one needs to define different hierarchical levels of abstraction. For example, application developers should not be involved with fault diagnosis and correction of middleware or configuration failures, although they need mechanisms to accurately detect if a perceived problem is due to a fault in lower-level services. At each hierarchical level, appropriate personal (e.g. application developer, middleware administrator and system support staff) should be responsible for dealing with faults. In this way, if a failure is detected on a higher layer, but its root cause is at a lower one, the corresponding staff should be activated to solve the problem. The challenge is to identify the right levels for this hand-on, allowing collaborative drilling-down in a controlled and effective manner. Ideally, the hand-off points should be narrow interfaces.

A possible way to address this challenge is providing automated test for each service in the grid [24]. Automated tests are key for enabling the staff solving a problem at layer n to determine whether the problem is their own or is at layer $n - 1$, without understanding how layer $n - 1$ works. Moreover, automated tests ease not only problem hand-on. After using the tests for the lower layer and concluding that the problem is at their own layer, support staff can use the tests for their own layer to expedite the problem isolation.

4 Conclusions

Bag-of-Tasks (BoT) applications are those parallel applications whose tasks are independent of each other. Despite their simplicity, BoT applications are used in a variety of scenarios. Moreover, due to the independence of their tasks, BoT applications can be successfully executed over widely distributed computational grids. Nevertheless, few users of BoT applications are currently using computational grids, despite the benefits grids can bring for BoT applications.

We have spent the last three years addressing this very issue. We have developed MyGrid [15] [25], an open-source software that enables the user to run BoT applications on whatever resources she has access to. MyGrid provides (i) a working environment that abstracts away the heterogeneity of the resources that form the grid, and (ii) efficient application scheduling. We have deployed MyGrid to some real users. They have got very excited about the solution, but invariably ask, “Where’s the grid for us to use?” This hinted us that providing just the middleware is not enough to solve this problem. BoT applications can make use of a lot of resources. The problem is that task of convincing administrators to yield access to resources is time-consuming and not scalable due to inherent trust issues. Therefore, we now believe that, besides the functionality provided by MyGrid, one also needs *automatic access granting* in order to truly enable the use of grids as the execution platform for BoT applications. We have a work-in-progress, called OurGrid [4], which aims to provide automatic access granting by creating a peer-to-peer community grid based on favors exchange.

Alas, the described functionality (grid working environment, application scheduling, and automatic access granting) faces a few practical deployment problems. In fact, we have deployed MyGrid in production to serve real users in the scientific community. Our efforts have unearthed four practical problems that hinder or limit the scope of grid computing for BoT applications. These are briefly summarized below.

Protocol heterogeneity: Implementing a grid for BoT applications would be relatively simple if standard job execution protocols were available on all platforms. Unfortunately, there is as yet no such standard grid infrastructure, and many protocols need to be used in order to remotely access resources. We address this problem by virtualizing resources, i.e. by hiding resources behind a standard interface, which we call *Processor Interface* or PI. The concept of polymorphic interfaces is a pillar in decoupling applications from the grid. However, it does not come for free. Writing a PI is very effort-intensive and new ways to access grid resources demand new coding.

Lack of end-to-end connectivity: One lesson learned is that today’s Internet connectivity is very complex and must be taken into account when designing any widely distributed system such as grid infrastructure. Firewalls, gateways and private IPs are commonplace. We did our best to supply work-around solutions such as tunneling support and application gateways, but the user typically needs to involve the system administrator to make solutions work.

Security issues: These include the establishment of trust, insuring message integrity and confidentiality, policy and privacy issues. Security turns out to

be even more difficult when executing BoT applications since the incentive to use “any and all resources available to get linear speedup” is strong. One needs to think on how to protect the application from a malicious resource, as well as how to protect the resource from a malicious application. Solutions are complex and still not fully resolved.

Fault diagnosis: Another yet unsolved problem. Even the most advanced grid middleware solution sadly lacks good fault diagnostic mechanisms. These mechanisms need to solve the basic tradeoff that, on the one hand, one does not normally want to see the details of how grid infrastructure does its magic while, on the other hand, detecting, diagnosing and fixing faults usually requires one to delve exactly into these details. A solution being investigated is to stratify fault-handling software so as to only expose what is absolutely necessary at each level.

It is important to stress the value of the feedback we get by deploying our solutions with real users. By deploying our solutions in practice we learn about problems we have not identified (such as automatic access granting), we realize that some problems we identified are not relevant in practice, and we become aware of the real administrative and technical obstacles in deploying our solutions. Due to the constant feedback of running in production, we are confident that we will have a good and practical solution for running BoT application on grids in a couple of years. Some of the practical four problems we have identified may be solved by technology itself (e.g. maybe we all adopt IPv6 some day). But it is very unlikely that all of them are solved this way. But this is not to say that these problems are unsolvable. It just means that a lot of effort is necessary to deploy real-life solutions.

Acknowledgments

We would like to thank Hewlett Packard and CNPq/Brazil for the financial support provided. This work could not have been done without it. Thanks to Fabrício Silva for the insightful remarks and for the great discussion about security. Thanks also to the OurGrid team for their great work.

References

- [1] D. Abramson, J. Giddy and L. Kotler. *High Performance Parametric Modeling with Nimrod/G: Killer Application for the Global Grid?* IPDPS'2000, pp. 520-528, Cancun Mexico, IEEE CS Press, 2000.
<http://ipdps.eece.unm.edu/2000/papers/Abramson.pdf>

- [2] D. Abramson, R. Buyya, J. Giddy. *A Computational Economy for Grid Computing and its Implementation in the Nimrod-G Resource Broker*. Future Generation Computer Systems Journal. Vol. 18. Elsevier Science. 2002.
- [3] D. Anderson, J. Cobb and E. Korpela. *SETI@home: An Experiment in Public-Resource Computing*. Communication of the ACM, vol. 45, no. 11, pp 56-61, November 2002.
- [4] N. Andrade, W. Cirne, F. Brasileiro, P. Roisenberg. *OurGrid: An Approach to Easily Assemble Grids with Equitable Resource Sharing*. Proc. 9th Workshop on Job Scheduling Strategies for Parallel Processing, June 2003.
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [5] Y. Aridor, D. Lorenz, B. Rochwerger, B. Horn, and H. Salem. *Reporting Grid Services (ReGS) Specification*. IBM Haifa Research Lab, draft-ggf-ogsa-regs-0.3.1, January 2003.
- [6] M. Baker, and G. Smith. *GridRM: A resource Monitoring Architecture for the Grid*. The Distributed Systems Group, University of Portsmouth, UK, June 2002.
- [7] A. Baratloo, M. Karaul, Z. Kedem, and P. Wyckoff. *Crarlotte: Metacomputing on the Web*. Proceedings of the 9th International Conference on Parallel and Distributed Computing Systems, 1996.
- [8] Jim Basney and Miron Livny. *Deploying a High Throughput Computing Cluster*. High Performance Cluster Computing, Rajkumar Buyya, Editor, Vol. 1, Chapter 5, Prentice Hall PTR, May 1999.
<http://www.cs.wisc.edu/condor/publications.html>
- [9] R. Buyya, S. Vazhkudai. *Compute Power Market: Towards a Market-Oriented Grid*. Proceedings of the First IEEE/ACM International Symposium on Cluster Computing and the Grid, CCGrid 2001.
- [10] H. Casanova, A. Legrand and D. Zagorodnov et al. *Heuristics for Scheduling Parameter Sweep Applications in Grid Environments*. In Proceedings of the 9th Heterogeneous Computing Workshop, pp 349-363, 2000.
<http://apples.ucsd.edu/hetpubs.html>
- [11] H. Casanova, J. Hayes, Y. Yang. *Algorithms and Software to Schedule and Deploy Independent Tasks in Grid Environments*. Workshop on Distributed Computing, Metacomputing, and Resource Globalization. Aussois, France. December 2002.
<http://grail.sdsc.edu/>

- [12] H. Casanova and F. Berman. *Parameter Sweeps on the Grid with APST*. In "Grid Computing: Making the Global Infrastructure a Reality", edited by Fran Berman, Geoffrey Fox and Tony Hey, John Wiley & Sons, April 2003.
<http://grail.sdsc.edu/>
- [13] W. Cirne e K. Marzullo. *Open Grid: A User-Centric Approach for Grid Computing*. Proceedings of the 13th Symposium on Computer Architecture and High Performance Computing, September 2001
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [14] W. Cirne and F. Berman. *A Comprehensive Model of the Supercomputer Workload*. Proc. WWC-4: IEEE 4th Annual Workshop on Workload Characterization, December 2001
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [15] W. Cirne, D. Paranhos, L. Costa, E. Santos-Neto, F. Brasileiro, J. Sauvé, F. Silva, C. Osthoff, C. Silveira. *Running Bag-of-Tasks Applications on Computational Grids: The MyGrid Approach*. Proceedings of the ICCP'2003: International Conference on Parallel Processing, October 2003.
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [16] K. Czajkowski, I. Foster, N. Karonis, C. Kesselman, S. Martin, W. Smith, S. Tuecke. *A Resource Management Architecture for Metacomputing Systems*. Proc. IPPS/SPDP '98 Workshop on Job Scheduling Strategies for Parallel Processing, pp. 62-82, 1998. <http://www.globus.org/research/papers.html>
- [17] C. De Rose, F. Blanco, N. Maillard, K. Saikoski, R. Novaes, O. Richard, B. Richard. *The Virtual Cluster: a Dynamic Environment for Exploitation of Idle Network Resources*. In 14th Symposium on Computer Architecture and High Performance Computing (SBAC-PAD'2002), 2002, Vitória, ES, Brazil.
- [18] I. Foster and C. Kesselman. *The Grid: Blueprint for a New Computing Infrastructure*. Morgan Kaufmann. 1998.
- [19] I. Foster, C. Kesselman, G. Tsudik, S. Tuecke. *A Security Architecture for Computational Grids*. Proc. 5th ACM Conference on Computer and Communications Security Conference, pp. 83-92, 1998.
- [20] I. Foster and C. Kesselman. *The Globus Project: A Status Report*. Proceedings of IPPS/SPDP'98 Heterogeneous Computing Workshop, pg. 4-18, 1998. <http://www.globus.org/research/papers.html>
- [21] I. Foster, C. Kesselman, J. Nick, S. Tuecke. *The Physiology of the Grid: An Open Grid Services Architecture for Distributed Systems Integration*. June 22, 2002. <http://www.globus.org/research/papers.html>

- [22] I. Goldberg, D. Wagner, R. Thomas, and E. A. Brewer. *A Secure Environment for Untrusted Helper Applications: Confining the Wily Hacker*. In Proceedings of the 1996 USENIX Security Symposium, 1996.
- [23] M. Litzkow, M. Livny, and M. Mutka. *Condor: A Hunter of Idle Workstations*. In Proceedings of the 8th International Conference of Distributed Computing Systems, pages 104-111, June 1988.
- [24] R. Medeiros, W. Cirne, F. Brasileiro, J. Sauvé. *Faults in Grids: Why are they so bad and What can be done about it?* Grid 2003: 4th International Workshop on Grid Computing, November 2003.
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [25] MyGrid Web Site. <http://dsc.ufcg.edu.br/mygrid>
- [26] D. Paranhos, W. Cirne, and F. Brasileiro. *Trading Cycles for Information: Using Replication to Schedule Bag-of-Tasks Applications on Computational Grids*. Europar'2003, Austria, 2003.
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [27] S. Smallen, W. Cirne and J. Frey et al. *Combining Workstations and Supercomputers to Support Grid Applications: The Parallel Tomography Experience*. Proc. of the HCW'2000 - Heterogeneous Computing Workshop. 2000.
<http://walfredo.dsc.ufcg.edu.br/resume.html#publications>
- [28] S. Smallen, H. Casanova, and F. Berman. *Applying Scheduling and Tuning to On-line Parallel Tomography*. Proceedings of Supercomputing 01, Denver, Colorado, USA, November 2001.
- [29] S. Son and M. Livny. *Recovering Internet Symmetry in Distributed Computing*. GAN'03 Workshop on Grids and Advanced Networks. Tokyo, Japan. May 12-15, 2003.
<http://www.cs.wisc.edu/~sschang/short+GAN03.pdf>
- [30] L. F. G. Sarmenta. *Sabotage-Tolerance Mechanisms for Volunteer Computing Systems*. ACM/IEEE International Symposium on Cluster Computing and the Grid (CCGrid'01), Brisbane, Australia, May 15-18, 2001.
- [31] P. Stelling, I. Foster, C. Kesselman, C. Lee, and G. Laszewski. *A Fault Detection Service for Wide Area Distributed Computations*. Proc. 7th IEEE Symp. On High Performance Distributed Computing, pp. 268-278, 1998.
- [32] J. R. Stiles, T. M. Bartol, E. E. Salpeter, and M. M. Salpeter. *Monte Carlo Simulation of Neuromuscular Transmitter Release Using MCell, a General Simulator of Cellular Physiological Processes*. Computational Neuroscience, pages 279-284, 1998.

- [33] B. Tierney, B. Crowley, D. Gunter, M. Holding, J. Lee, and M. Thompson. *A Monitoring Sensor Management System for Grid Environments*. Proc. IEEE High Performance Distributed Computing Conference (HPDC-9), Aug. 2000.
- [34] J. Weissman. *Fault Tolerant Computing on the Grid: What are My Options?* Technical Report, University of Texas at San Antonio, 1998.